

Article

Pre-During-After Software Development Documentation (PDA-SDD): A Phase-Based Approach for Comprehensive Software Documentation in Modern Development Paradigms

Abdullah A. H. Alzahrani

Computers Department Engineering and Computers College—Alqunfuda, Umm Al Qura University, Makkah 24381, Saudi Arabia; aahzahrani@uqu.edu.sa

Abstract

Persistent challenges in software documentation, particularly limitations in generality, simplicity, and efficiency of existing models, impede effective software development. To address these, this research proposes a novel phase-based and holistic software documentation model (PDA-SDD). This model was subsequently evaluated using a digital questionnaire distributed to 150 software development and documentation experts, achieving a 48% response rate ($n = 72$). The evaluation focused on assessing the proposed model's generality, simplicity, and efficiency. Findings indicate that while certain sub-models (e.g., SRSD, RLD) were positively received across all criteria and the overall model demonstrated strong perceived generality and efficiency in specific aspects, areas for improvement were identified, particularly regarding terminological consistency and user-friendliness. This study contributes to the understanding of the complexities in achieving a universally effective software documentation model and highlights key considerations for future research and development in this critical area of software engineering.

Keywords: software engineering; software systems documentation; software maintenance; software quality; software development



Academic Editor: Yan Liu

Received: 2 August 2025

Revised: 31 August 2025

Accepted: 4 September 2025

Published: 9 September 2025

Citation: Alzahrani, A.A.H.

Pre-During-After Software Development Documentation (PDA-SDD): A Phase-Based Approach for Comprehensive Software Documentation in Modern Development Paradigms. *Computers* **2025**, *14*, 378. <https://doi.org/10.3390/computers14090378>

Correction Statement: This article has been republished with a minor change. The change does not affect the scientific content of the article and further details are available within the backmatter of the website version of this article.

Copyright: © 2025 by the author. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Software documentation is a fundamental and indispensable artifact within the ecosystem of software development. It acts as the linchpin for disseminating critical information across a multifaceted spectrum of stakeholders. This diverse cohort includes not only the core development team—such as architects, programmers, and quality assurance engineers—but also end-users and specialized personnel responsible for long-term maintenance. Creating comprehensive and effective documentation is a foundational prerequisite for a shared understanding of a system's architecture, functionalities, and underlying principles. This practice facilitates seamless collaboration, ensures long-term maintainability, and ultimately enhances the overall quality of software deliverables [1–3].

Despite its widely recognized importance, the processes for generating and maintaining software documentation are frequently beset by a multitude of persistent challenges [4,5]. These difficulties often stem from the inherent complexity of contemporary software systems, which are marked by intricate architectural designs, rapidly evolving functionalities, and complex interdependencies. The significant heterogeneity in the information requirements of diverse stakeholder groups further exacerbates these challenges. A systematic examination of the scholarly literature reveals a discernible categorization

of these persistent challenges along three principal dimensions: generality, simplicity, and efficiency [6,7].

Firstly, many existing software documentation models tend towards functional or structural specialization, often concentrating their scope on discrete facets like Application Programming Interfaces (APIs) or requirements specifications (SRS) [8]. Consequently, these specialized models often lack the generality needed for holistic documentation. Secondly, the perceived simplicity and overall usability of established documentation processes are often undermined by stringent requirements for specialized competencies, which acts as a significant barrier for stakeholders [9]. Finally, the efficiency of various proposed models varies considerably. Many models demand significant temporal and cognitive resources, often leading to incomplete, outdated, or neglected documentation due to perceived overhead [10].

This research explicitly addresses these limitations. The central objective of this study is to explore the potential of a new model that prioritizes generality, simplicity, and efficiency. By offering a holistic, phase-based, and integrated framework, our model ensures broad applicability for diverse stakeholders throughout the entire software development process. To achieve this, our investigation began with a rigorous analysis of existing models to evaluate their strengths and weaknesses against the identified core challenges. The insights from this analysis then served as the foundation for the development of our novel theoretical model: the Pre-During-After Software Development Documentation (PDA-SDD) Model. We subsequently evaluated its efficacy and practical applicability through a rigorous process of expert validation involving detailed feedback from experienced professionals.

2. Methodology and Research Questions

Effective software documentation consistently faces challenges related to its generality, simplicity, and efficiency. Building upon a thorough analysis of these persistent challenges in current practices, this study formulated specific research questions to guide the development and evaluation of the PDA-SDD model.

The following research questions guided this investigation:

RQ1: What are the essential characteristics required for a comprehensive software documentation model to effectively address the identified challenges of generality, simplicity, and efficiency across diverse software development contexts and stakeholder needs?

RQ2: To what extent does the proposed PDA-SDD model exhibit the desired attributes of generality, simplicity, and efficiency, as perceived by software development and documentation experts?

To address RQ1, a comprehensive analysis of existing software documentation models and studies was undertaken. The objective was to critically evaluate whether these models effectively address the identified challenges of generality, simplicity, and efficiency, thereby pinpointing the essential characteristics needed for a superior model. The findings of this analysis are presented in the subsequent Section 3.

RQ2 was addressed by developing the theoretical PDA-SDD model, which is designed to overcome the aforementioned limitations. To validate the model and answer RQ2, it was conceptually evaluated by presenting it to experienced software development and documentation professionals for their expert opinions. The evaluation specifically focused on assessing the model in terms of its perceived generality, simplicity, and efficiency.

A digital questionnaire served as the primary method used to evaluate the model for RQ2. The questionnaire was targeted to a specific group of software development and documentation experts, and it included questions related to the generality, simplicity, and efficiency of the proposed model. A Likert scale was employed, with ratings ranging from 1 (strongly disagree) to 5 (strongly agree), to quantify and interpret the responses.

Google Forms was used to create the questionnaire, which was then distributed to the target group via email and social media. A total of 150 experts from various countries and involved in different software projects were invited to participate. This sample size was selected to ensure a sufficiently large and representative pool of professional perspectives to yield statistically meaningful results, mitigating the risk of bias from a small or un-diverse group. A response rate of 48% was achieved, resulting in a total of 72 expert participants.

2.1. Survey Participant Characteristics and Methodological Considerations

The evaluation of the PDA-SDD model was conducted using survey responses from 72 participants who self-identified as experienced professionals in software development or documentation. The demographic and professional background distribution of these participants is meticulously detailed in Table 1.

Table 1. Participant Demographics and Professional Background.

Category	Sub-Category	Count	Percentage
Gender Distribution	Male	46	63.90%
	Female	26	36.10%
Job Title Distribution	Software Analyst	11	15.30%
	Software Designer	2	2.80%
	Software Developer	7	9.70%
	Software Tester	14	19.40%
	Software Quality Auditor	9	12.50%
	Software Maintainer	8	11.10%
	Software Project Manager	10	13.90%
	Software Project Coordinator	4	5.60%
	Software Engineer	7	9.70%
Qualification Level	Diploma	28	38.90%
	BSc.	12	16.70%
	MSc.	16	22.20%
	Ph.D.	16	22.20%
Years of Experience	1 to 5 years	28	38.90%
	6 to 10 years	10	13.90%
	11 to 15 years	8	11.10%
	16 to 20 years	14	19.40%
	Over 20 years	12	16.70%
Total Participants		72	100%

A key consideration in this study is that participant expertise was entirely self-reported. This introduces a potential source of variability in the actual depth of individual expertise, as no external cross-validation was employed. While this approach is common in exploratory survey research, future investigations could enhance data robustness by incorporating more rigorous participant screening.

Furthermore, it is crucial to acknowledge the potential for inherent biases in survey-based research. For instance, selection bias may have influenced the participant pool if recruitment strategies disproportionately attracted individuals with a pre-existing interest in documentation or a favorable predisposition towards novel models. Response bias, such as social desirability bias, might also have led participants to provide responses perceived as more academically or professionally desirable rather than entirely candid reflections of their true opinions. Lastly, despite participants originating from diverse organizational contexts and representing a range of job titles, a degree of participant background homogeneity could still exist within the self-reported “expert” cohort. This may limit the broader generalizability of the findings to organizations or teams employing substantially different approaches.

Acknowledging these participant characteristics and potential methodological biases ensures a transparent account of the study’s limitations and concurrently informs the design of future, more robust empirical investigations aimed at validating the PDA-SDD model.

2.2. Research Methodology Flowchart

To provide a clear and intuitive understanding of the research design, Figure 1 presents a comprehensive flowchart of the research methodology. This diagram visually illustrates the complete sequence of steps and the overall iterative process that guided our work, from the initial systematic literature review to the survey-based evaluation phases. This visual aid offers a concise overview of how the research was conducted from problem identification through to the final analysis and conclusions.

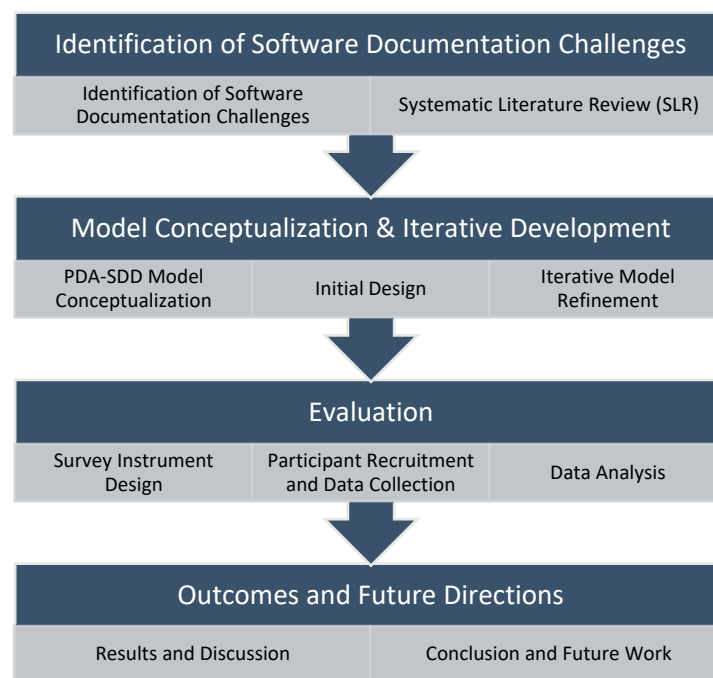


Figure 1. Research Methodology Flowchart.

3. Related Work and Background

Software documentation is a fundamental artifact throughout the software development lifecycle, from pre-coding to post-deployment. It captures a system’s rationale, design, functionality, and operation [1,2]. Despite its critical role in communication and collaboration, many deployed systems have inadequate, ambiguous, or obsolete documentation [3,4]. This often stems from the perceived burden of technical writing and the significant time and resources required [5–10]. This perception often leads to prioritizing coding over documentation, creating critical gaps that hinder future development and maintenance [11–17].

Foundational artifacts, such as the Software Requirements Specification (SRS) and the User’s Manual, are of paramount importance [11]. The SRS, crafted early in the system lifecycle, is considered pivotal for system success, as it articulates functional and non-functional requirements [1,2]. While international organizations like IEEE and ISO have developed comprehensive guidelines for such documents [18–26], concerns persist regarding human error and subjective interpretation during authoring. This underscores the need for more rigorous and potentially automated approaches to documentation beyond traditional standards alone.

Extensive research [27–43] has leveraged standardized models from organizations like IEEE and ISO to improve requirements engineering, enhancing processes through features such as traceability mechanisms and serving as benchmarks for quality and compliance verification. Similarly, the End User License Agreement (EULA) is a critical software artifact for which organizations and researchers have developed standardized models. Its significance, particularly within the Open Source community, is further underscored by scholarly work focused on automating the detection of EULA non-compliance [44–46].

While robust documentation is vital, existing tools and methodologies are often specialized and lack a truly holistic solution. For example, tools like IBM DOORS [47,48], GitBook [49,50], and Stoplight [51] offer capabilities for specific aspects such as requirements management or collaborative content creation. Similarly, Model-Based Systems Engineering (MBSE) tools [52] assist with integrated design. However, these approaches frequently fall short of providing a universally general, simple, and efficient solution for managing all facets of documentation across the entire development lifecycle and for all diverse stakeholders. They often remain siloed, require significant integration effort, or lack the integrated conceptual model needed for comprehensive documentation. This highlights a persistent gap for a comprehensive and adaptable framework.

This research builds upon a broader scholarly observation, including a comprehensive systematic review [15] by the authors, which revealed a notable lack of sustained attention on holistic software documentation processes. There is a consensus that creating effective documentation requires specialized knowledge to address pervasive inconsistencies and missing information. The seminal work of Theunissen et al. [16,17] introduced a comprehensive framework for systematic knowledge transfer, empirically evaluated within software maintenance. While their work offers valuable insights, our novel PDA-SDD model distinguishes itself by directly focusing on the documentation process itself across the entire system lifecycle, emphasizing our three key criteria.

The Evolving Landscape of Software Documentation

Modern software development, characterized by agile and DevOps methodologies, has fundamentally shifted how documentation is managed. It is no longer a static artifact but a dynamic, continuous process [53].

In Agile development, documentation must function as a living, iteratively updated artifact that balances comprehensiveness with rapid development velocity [54]. In practice, this often means creating “just-enough” documentation and prioritizing a working product over extensive written materials [55]. DevOps practices further necessitate documentation that supports automation and seamless deployment, leading to “documentation as code” via versioning and automated publication within CI/CD pipelines [56,57]. A recent example of this is the development of frameworks like Architecture as Code, which treat architectural definitions as machine-readable assets to automate compliance and reduce bottlenecks [58].

Furthermore, Artificial Intelligence (AI), particularly Large Language Models (LLMs), has emerged as a major force in the field. Recent research shows that AI is increasingly used to automate the generation, updates, and maintenance of documentation, enhancing efficiency and consistency [59,60]. AI-powered tools are now integrated into development environments to generate production-ready code from prompts and to create reference documentation automatically [61]. However, the use of LLMs also introduces challenges, such as the potential for generating inaccurate or “hallucinated” content, which highlights the continued need for human oversight and rigorous evaluation [62,63].

Addressing these evolving demands and the persistent gaps in existing solutions, the Pre-During-After Software Development Documentation (PDA-SDD) model is con-

ceptualized as a holistic framework that bridges traditional principles with contemporary realities. Its structured, phase-based approach inherently supports dynamic documentation practices. By utilizing living documents, rigorous versioning, and centralized data management, PDA-SDD seamlessly integrates with iterative development cycles, automated pipelines, and future AI-driven enhancements, offering a comprehensive and adaptable solution for modern software documentation.

4. PDA-SDD Model (The Theoretical Framework)

Building upon the comprehensively documented limitations of existing documentation approaches and specialized tools [47–52] and explicitly recognizing the dynamic and evolving demands of modern software development practices [53,54,56,57,64–67], this section introduces the Pre-During-After Software Development Documentation (PDA-SDD) model. The PDA-SDD model is conceptualized as a truly holistic, universally applicable, simple, and highly efficient framework specifically designed to guide and streamline software documentation throughout the entire software development lifecycle, offering distinctive advantages over current fragmented methodologies. Its primary objective is to ensure comprehensive coverage for all diverse stakeholders, from developers and project managers to end-users and maintenance teams. By adopting a structured, phase-based approach, PDA-SDD directly addresses the persistent research gap for an integrated, adaptable, and pragmatic documentation solution that transcends the inherent limitations of fragmented, siloed tools and phase-specific methodologies. It seeks to transform documentation from an afterthought into a continuous, valuable asset, integrated seamlessly into every stage of development.

4.1. Rationale and Development Process

The core rationale behind the conception of the PDA-SDD model stems directly from the persistent research gap explicitly identified in Section 3: the notable absence of a single, integrated framework capable of offering simplicity, generality, and efficiency in comprehensively managing all facets of software documentation across its entire lifecycle. Current documentation practices often rely on a disparate collection of specialized tools, each powerful within its niche but fundamentally lacking a cohesive overarching strategy that supports documentation's evolution and integration. For instance, while IBM DOORS [47,48] provides robust capabilities for requirements traceability in large, complex, and often highly regulated projects, its primary focus remains on requirements management rather than comprehensive, evolving documentation across all project phases. Similarly, modern platforms like GitBook [49] and Stoplight [51] excel at facilitating collaborative content creation, version control, and automated publishing, particularly for API documentation [50,60] or internal knowledge bases, yet they do not inherently provide the structured guidance for capturing the full spectrum of development rationale, design decisions, and operational details from project inception to post-deployment. Even sophisticated Model-Based Systems Engineering (MBSE) tools [52] integrate design and analysis with documentation by linking artifacts to a central model, but their adoption often requires specialized expertise, significant upfront investment, and a particular development paradigm, thereby limiting their universal applicability and simplicity for all project types and team sizes. These existing solutions, while undoubtedly valuable, tend to operate in silos, demand significant integration overhead, or fail to provide the unified, holistic view of documentation's evolution needed from initial conceptualization (pre-implementation) through active development (during-implementation) to ongoing post-deployment maintenance. The critical imperative for a model that actively bridges these functional and temporal gaps, enabling documentation to function as a continuous, living artifact rather

than a static, burdensome deliverable, served as the primary driving force for the ideation and rigorous development of PDA-SDD.

The development of the PDA-SDD model followed a rigorous, iterative, and informed process, meticulously grounded in a thorough understanding of both perennial documentation challenges and the dynamic requirements of contemporary software engineering paradigms. This systematic approach ensured that the model's design was not merely theoretical but also pragmatically addresses real-world industry needs and operational complexities.

1. **Comprehensive Literature Review and Gap Identification:** The development process commenced with an exhaustive literature review (as detailed in Section 3), which systematically identified prevalent documentation deficiencies, analyzed existing best practices, and critically assessed the limitations of current solutions and tools. This initial foundational phase allowed for a precise articulation of the research gap—the urgent need for a simplified, generalized, and efficient model for managing documentation across the entire software lifecycle. This systematic analysis directly informed PDA-SDD's core conceptualization: shifting the perception of documentation from a discrete, isolated task performed at specific project milestones to an integral, continuous process that requires structured, phased management across the entire software development continuum.
2. **Derivation from Established Standards and Principles:** Foundational principles for document structure, quality attributes, and comprehensive content coverage were rigorously derived from established international documentation standards and guidelines. Prominent examples include the comprehensive guidelines from the Institute of Electrical and Electronics Engineers (IEEE) and the International Organization for Standardization (ISO) [18–26], which provided robust templates and best practices for critical documents such as the Software Requirements Specification (SRS), Software Design Description (SDD), and user manuals. These standards informed PDA-SDD's inherent emphasis on clarity, consistency, completeness, and maintainability of documentation artifacts, ensuring their long-term value.
3. **Analysis of Modern Development Methodologies:** A critical analysis of contemporary software development methodologies, most notably Agile and DevOps [53,54,56,57,64,65] highlighted the imperative for flexible, dynamic, and “living” documentation. This analysis underscored the necessity for documentation that can adapt seamlessly to rapid iterations, frequent changes, and automated workflows inherent in modern development environments. Insights gleaned from DevOps practices, specifically the “documentation as code” paradigm, profoundly influenced PDA-SDD's design to inherently support key features such as versioning, automated documentation generation, and seamless integration into Continuous Integration/Continuous Deployment (CI/CD) pipelines. This ensures documentation remains synchronized with the evolving codebase and accessible within development toolchains.
4. **Phased Structure Emergence and Iterative Refinement:** The model's three-phase structure—Pre-implementation, During-implementation, and After-implementation—logically emerged from this integrated analysis. These phases represent a natural and intuitive progression aligned with typical software lifecycle stages, ensuring that documentation activities are synchronized and intrinsically linked with development activities. Each phase was meticulously designed to identify critical documentation artifacts (e.g., requirements, design choices, test plans, user guides), the associated processes for their creation and maintenance, and the key stakeholder involvement required at each stage. For instance, the Pre-implementation phase focuses on foundational documents like detailed requirements and initial architectural visions; the During-implementation phase emphasizes design decisions, technical specifications,

test plans, and code-level documentation; and the After-implementation phase then encompasses deployment guides, user manuals, and maintenance logs. The entire PDA-SDD model was continually refined through multiple conceptual iterations, internal discussions, and feedback loops. This iterative process ensured its steadfast adherence to its guiding principles of simplicity, generality, and efficiency, which are core to its unique advantages and broad applicability across diverse teams. The ultimate aim of this rigorous development process was to produce a framework that is both theoretically sound, grounded in established best practices, and practically applicable for a wide array of software development teams and contexts seeking to improve their documentation practices holistically.

4.2. Core of the Model

The Pre-During-After Software Development Documentation (PDA-SDD) Model, shown in Figure 2, presents a systematic framework for managing documentation throughout the software development lifecycle. Organized into three primary phases—Pre-implementation, During-implementation, and After-implementation—the PDA-SDD Model outlines essential documentation requirements while accommodating optional additions.

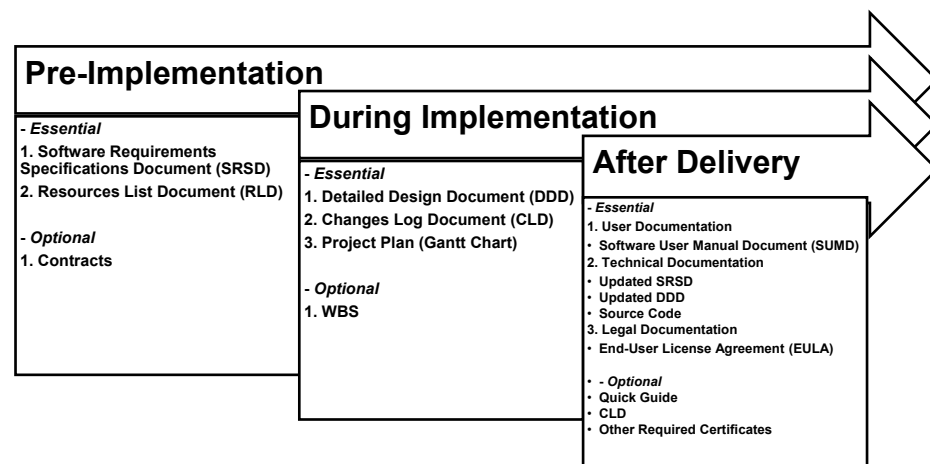


Figure 2. PDA-SDD Model.

Pre-implementation mandates the inclusion of a Software Requirements Specifications Document (SRSD) and a Resources List Document (RLD). Optional components may encompass relevant contracts.

During-implementation, essential documents include a Detailed Design Document (DDD), a Change Log Document (CLD), and a comprehensive project plan. A Gantt Chart is recommended for project planning, while a Work Breakdown Structure (WBS) may be incorporated optionally.

After-implementation, essential documentation includes user documentation (Software User Manual), technical documentation (updated SRSD, DDD, and Source Code), and legal documentation (End User License Agreement). Optional components may include a Quick Guide, updated CLD, and relevant certifications.

The PDA-SDD Model leverages a versioning system to ensure document consistency and completeness throughout the development process. Centralized data storage facilitates the prevention of duplication and the maintenance of document integrity.

4.3. Main Documentations

This section delves into the core components of the proposed documentation model, providing detailed explanations for each. To facilitate comprehensive coverage, this section is divided into six subsections representing the most critical aspects of the model.

4.3.1. Software Requirements Specifications Document (SRSD)

The SRSD serves as the foundational element of the proposed documentation model, warranting meticulous attention. As the cornerstone of the system, it underpins all subsequent documentation and system implementation efforts. Contracts are established based on the SRSD, and all system documents are derived from it.

The SRSD employs a coding system that avoids excessive detail while capturing essential information about the system, including its scope, objectives, and main functions. As illustrated in Table 2, the coding system assigns a unique code to each system requirement, categorizing them into functional requirements (SFR) and non-functional requirements (SNFR).

Furthermore, the coding system subdivides functional and non-functional requirements into sub-requirements, allowing for granular coding of individual system requirements. This ensures clear identification and reference to each requirement throughout the design and documentation processes. Figure 3 depicts the structure of the coding system and its relationship to the various requirement types. The expected structure of the SRSD is outlined in Appendix A.

Table 2. SRSD codes.

Codes	Components	Requirement Type	Sub-Type
SGI	General Info		
SGI-S	Scope		
SGI-OJ	Objectives		
SGI-MF	Main Functions		
SFR	Functional Requirements	IO	
SFR-IO			
SFR-IODE			Data Entry
SFR-IODO		Processing Requirements	Data Output
SFR-IOR			Reporting
SFR-PR			
SFR-PRC		Business Rule	Calculation
SFR-PRDM			Decision Making
SFR-PRDP			Data Manipulation
SFR-BR		Security Requirements	Constraints
SFR-BRC			Validation
SFR-BRV			Workflow
SFR-BRW		Integration Requirements	
SFR-SR			Authentication
SFR-SRAN			Authorization
SFR-SRAZ			Access Control
SFR-SRAC			
SFR-IR			Interface
SFR-IRI			Data Exchange
SFR-IRDX			Interoperability
SFR-IRIN			

Table 2. Cont.

Codes	Components	Requirement Type	Sub-Type
SNFR	Non-Functional Requirements	Performance	Response Time Throughput Scalability
SNFR-P			
SNFR-PRT			
SNFR-PT		Usability	Ease of Use Efficiency Aesthetics
SNFR-PS			
SNFR-U			
SNFR-UEU		Security	Confidentiality Integrity Availability
SNFR-UE			
SNFR-UA			
SNFR-S		Reliability	Availability Accuracy Robustness
SNFR-SC			
SNFR-SI			
SNFR-SA		Maintainability	Modifiability Testability Portability
SNFR-R			
SNFR-RAV			
SNFR-RAC			
SNFR-RR			
SNFR-M			
SNFR-MM			
SNFR-MT			
SNFR-MP			

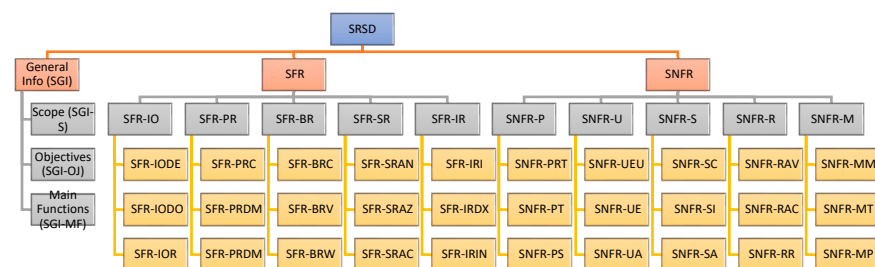


Figure 3. SRSD structure.

The SRSD is anticipated to deliver a set of well-defined requirements, classified by their coding system. This clarity and precision contribute to the removal of ambiguity, facilitate future modifications and updates, and establish linkages between requirements and subsequent documentation.

4.3.2. Resources List Document (RLD)

The Resources List Document (RLD) constitutes a pivotal document that empowers project managers and other stakeholders involved in project planning to assess their capabilities and establish a solid foundation for the pre-implementation stage. This document encompasses several sections, including two fundamental categories: human resources and available equipment.

As depicted in Figure 4, the RLD comprises two primary components: human resources and equipment. The human resources section is further divided into two sub-sections: a summary of human resources, which outlines job titles and corresponding quantities, and detailed information on individual team members, including names, job titles, experience, qualifications, and hourly rates.

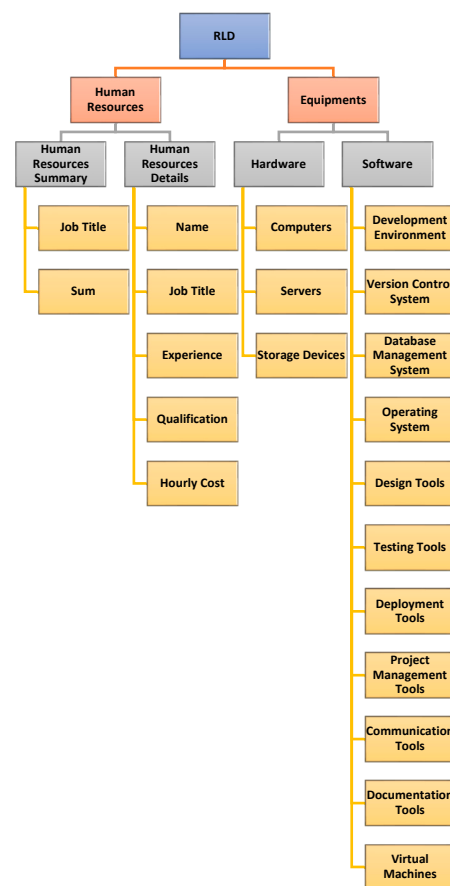


Figure 4. RLD structure.

Regarding the equipment section of the RLD, it is noteworthy that the equipment is categorized into hardware and software components. Within each category, three key elements are identified: total equipment, equipment types. For a comprehensive overview of the proposed and anticipated structure of this document, please refer to Appendix A.

4.3.3. Detailed Design Document (DDD)

The Detailed Design Document (DDD) constitutes a cornerstone of the proposed model within the During-implementation component of the PDA-SDD Model. As illustrated in Figure 5, the DDD encompasses several sections.

The initial section, the traceability matrix, facilitates the linkage between modules within the DDD and the corresponding requirements outlined in the SRSD. The second section addresses system architecture, comprising two primary components: a general description of the system architecture, often represented through a diagrammatic representation, and an explanation of the employed programming languages, patterns, and tools.

The third section of the DDD focuses on data design, including: (1) Description of objects and their relationships (e.g., using an ER diagram). (2) Selection of appropriate data structures. (3) Database schema. The fourth section of the DDD incorporates graphical representations of user interfaces and APIs for intersystem communication.

A pivotal section of the DDD is the Module design section. For each module within the system, a subsection is created to detail that includes Module responsibilities, structure, interactions, Algorithms and descriptions, and Selected data structures. Class Diagrams and Data Flow Diagrams may be employed to represent module structure whereas Sequence Diagrams and Activity Diagrams, or other UML diagrams may be employed to represent module interactions. For a comprehensive overview of the proposed and anticipated structure of this document, please refer to Appendix A.

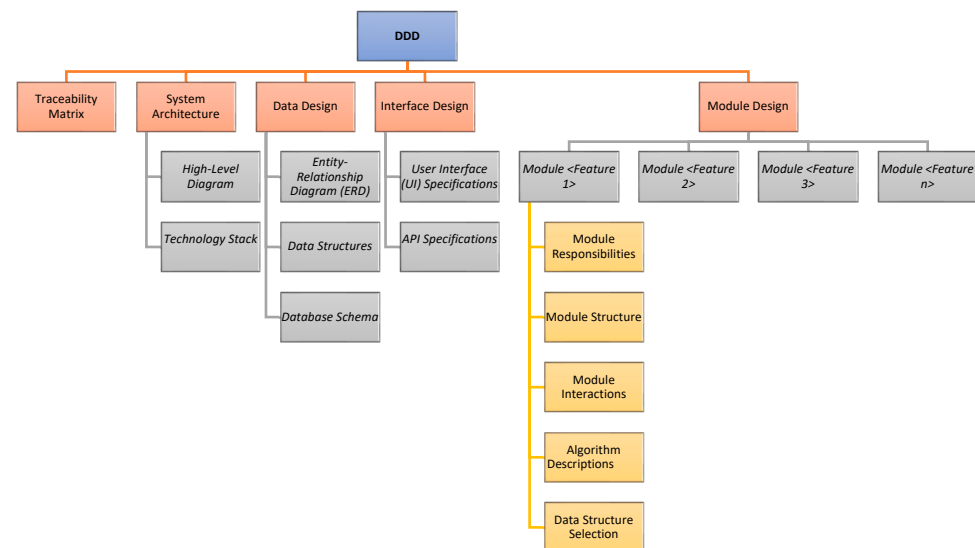


Figure 5. DDD structure.

4.3.4. Change Log Document (CLD)

The CLD (Change Log Document) serves as a pivotal component within the PDA-SDD (Pre-During-After Software Development Documentation) model. Its primary function is to meticulously chronicle the evolution of the entire project, encompassing modifications, additions, and accomplishments across various artifacts such as the SRSD (Software Requirements Specification Document), DDD (Detailed Design Document), and the source code itself.

The CLD is organized into distinct sections, each corresponding to a primary function outlined in the SRSD, beginning with SGI-MF (System General Info-Main Function). Consequently, the number of sections within the CLD directly aligns with the number of main functions specified in the SRSD.

As illustrated in Figure 6, the CLD structure is characterized by a feature-based organization, where each feature corresponds to a main function from the SRSD. Within each feature, a series of Logs, identified by unique incremental numbers, are employed to record specific activities related to the function. Each Log comprises eight essential elements that provide a comprehensive account of the activity.

The initial subsection of the Log section is dedicated to significance, delineating whether the activity is major or minor and its classification within the requirements, design, or code domains. The subsequent subsection, results, documents the outcome of the activity, including the success or failure of implementation and testing, as well as any approved design or requirement modifications. The date and time of the activity are then recorded in the following subsection.

To provide further context and detail regarding the activity, the Change Summary and Detailed Change subsections are included. Additionally, a References subsection is designated to link the change to any relevant requirements. If an electronic ticketing system is utilized, an Issue Tracking System subsection is incorporated to reference associated ticket numbers. Finally, the Log is concluded with the name of the individual responsible for performing or recording the activity. For a comprehensive overview of the proposed and anticipated structure of this document, please refer to Appendix A.

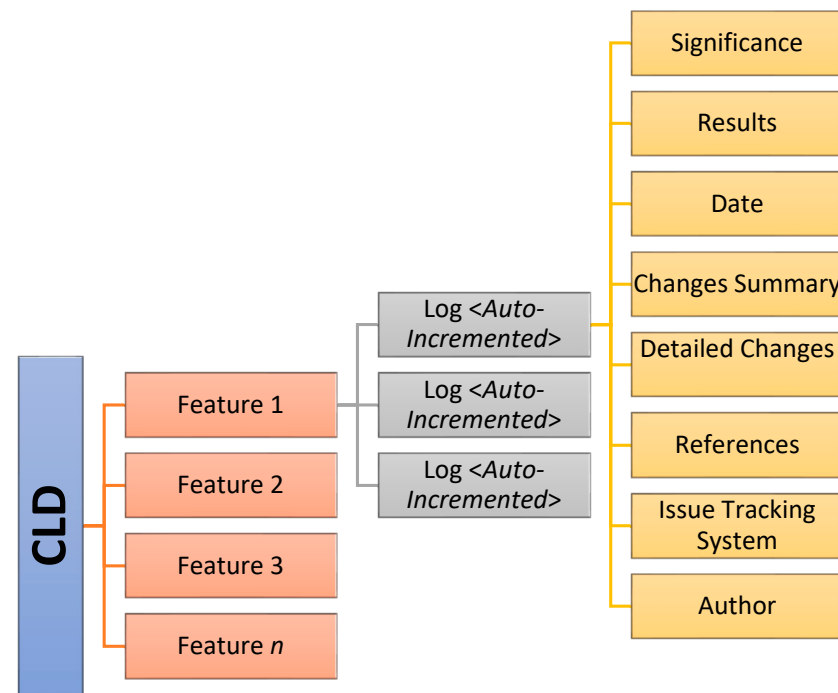


Figure 6. CLD structure.

4.3.5. Software User Manual Document (SUMD)

The System User Manual and Documentation (SUMD) serves as a pivotal resource for end-users, offering comprehensive guidance on the system's functionality, terminology, and operation. It facilitates a smooth transition into system usage, outlining both basic and advanced features. Furthermore, the SUMD provides a valuable troubleshooting resource, addressing potential errors and offering solutions.

As depicted in Figure 7, the proposed PDA-SDD model incorporates a structured SUMD framework comprising eight essential sections. The introductory section provides a general overview of the system, its scope, and linguistic conventions. Subsequently, the document addresses system installation, outlining prerequisites and activation procedures. A section dedicated to basic functions and interfaces is followed by a more in-depth exploration of advanced features. To assist users in resolving issues, a troubleshooting section is included, covering common problems, expected error messages, and technical support options. Finally, the SUMD concludes with a glossary of terms and any relevant attachments in an appendix section. For a comprehensive overview of the proposed and anticipated structure of this document, please refer to Appendix A.

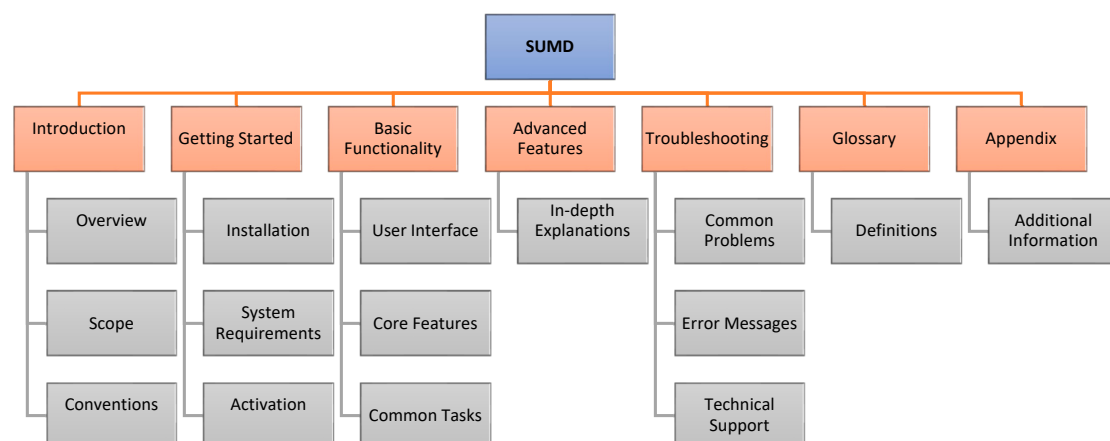


Figure 7. SUMD structure.

4.3.6. End User License Agreement (EULA)

The End-User License Agreement (EULA) serves as a foundational legal document for any software system. While the specific content of EULAs can vary widely, a core set of elements commonly recurs. The PDA-SDD model proposes a standardized EULA structure that encompasses these essential components.

As illustrated in Figure 8, the PDA-SDD model outlines a nine-section EULA framework. These sections include: Grant of License, Ownership, Restrictions, Disclaimer of Warranty, Indemnity, Termination of Agreement, Governing Law and Jurisdiction, Entire Agreement, and Severability. This standardized structure provides a valuable reference point for drafting and understanding EULAs.

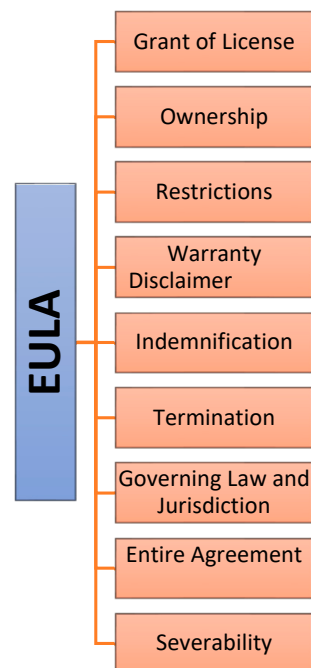


Figure 8. EULA Document structure.

It is important to note that the specific content and emphasis of EULAs can differ depending on factors such as the nature of the software, the parties involved, and the applicable legal jurisdiction. However, the nine elements identified in the PDA-SDD model represent a fundamental foundation for most EULAs. Given the legal significance of EULAs, it is strongly recommended that they be reviewed and approved by legal professionals. For a comprehensive overview of the proposed and anticipated structure of this document, please refer to Appendix A.

4.4. Distinctive Advantages of the PDA-SDD Model

The Pre-During-After Software Development Documentation (PDA-SDD) model offers several distinctive advantages that set it apart from existing documentation approaches and tools, directly addressing the persistent challenges and research gaps identified in Section 3. While drawing upon established best practices, PDA-SDD's novelty and superiority stem from its integrated, holistic, and systematic framework:

1. **Holistic Lifecycle Coverage:** Unlike many existing tools or methodologies that focus on specific stages (e.g., requirements management) or types of documentation (e.g., API documentation), PDA-SDD provides a comprehensive, phase-based framework (Pre-implementation, During-implementation, After-implementation). This ensures that critical system knowledge, design rationale, and operational details are systematically

- captured and maintained throughout the entire software development and deployment lifecycle, from initial conceptualization to post-implementation maintenance.
2. **Unified Framework for Diverse Documentation:** PDA-SDD offers a single, coherent conceptual model for managing all essential documentation artifacts. This contrasts with the fragmented landscape where organizations often rely on a disparate collection of siloed tools and ad hoc practices, leading to inconsistencies, redundancy, and knowledge gaps. PDA-SDD mandates specific, interlinked deliverables for each phase, promoting a unified and consistent approach.
 3. **Emphasis on Simplicity, Generality, and Efficiency:** These core principles are foundational to PDA-SDD's design. The model aims to simplify the documentation process, reducing the perceived burden on development teams, thereby encouraging consistent adoption. Its generality ensures adaptability across various project sizes, technological stacks, and organizational structures, unlike specialized tools that may require significant customization or expertise. The focus on efficiency optimizes resource utilization, minimizing overhead for documentation creation and maintenance.
 4. **Integration with Modern Development Paradigms:** PDA-SDD is specifically designed to align with contemporary software development methodologies such as Agile, DevOps, and CI/CD pipelines. Its inherent support for living documents, rigorous versioning, centralized data management, and the “documentation as code” paradigm ensures that documentation remains dynamic, accurate, and seamlessly integrated into iterative development cycles and automated workflows. This is a crucial differentiator from traditional static documentation models.
 5. **Enhanced Stakeholder Collaboration and Communication:** By clearly defining phase-specific artifacts and outlining stakeholder involvement, PDA-SDD fosters improved communication and collaboration among all project participants—from developers and testers to project managers, end-users, and maintenance teams. This systematic approach ensures that relevant information is accessible, up-to-date, and tailored to the needs of diverse audiences throughout the system's evolution.
 6. **Foundation for Automation and Future Technologies:** The structured and consistent nature of PDA-SDD's mandated artifacts provides a robust foundation for future integration with advanced automation tools and Artificial Intelligence (AI) technologies, such as Large Language Models. This inherent structure facilitates automated document generation, consistency checks, and intelligent knowledge retrieval, further enhancing efficiency and accuracy.

In summary, PDA-SDD's distinctive advantage lies not merely in its composite nature, but in its strategic integration of best practices into a holistic, phase-driven, and adaptable framework that addresses the full spectrum of software documentation challenges across the entire lifecycle, setting a new standard for comprehensive and efficient documentation management.

4.5. Comparative Analysis with Existing Models

To further illustrate the distinctive advantages and novelty of the PDA-SDD model, Table 3 provides a direct side-by-side comparison with prominent existing documentation approaches and frameworks. This comparison highlights key differentiators across critical criteria, demonstrating how PDA-SDD integrates and extends current best practices into a more holistic, adaptable, and efficient solution for comprehensive software documentation.

Table 3. Comparative analysis of documentation models.

Feature/Criterion	Traditional Documentation Standards (e.g., IEEE 830 for SRS) [18–26]	Agile/Lightweight Documentation Principles (e.g., “Just-in-Time”) [53,54,56,57,64,65]	Specialized Documentation Tools (e.g., Requirements Management, API Docs) [47–52]	PDA-SDD Model
Scope of Lifecycle Coverage	Specific phases/ documents (e.g., requirements); less continuous.	Limited to current iteration; fragmented without overall strategy.	Niche-specific (e.g., requirements, APIs); rarely integrated across full lifecycle.	Comprehensive SDLC (Pre-, During-, After-); emphasizes continuous, living documentation.
Generality and Adaptability	Rigid for formal projects; less adaptable for diverse needs.	High for small, co-located teams; less structured for large/complex projects.	High for niche; limited beyond specialty; often requires expertise.	Highly general across project types, sizes, tech stacks, and organizational structures.
Simplicity and Efficiency	Complex, prescriptive; high overhead.	Prioritizes simplicity (reduces overhead); can lead to documentation gaps/inconsistencies.	Varies by tool; efficient for niche, but high integration overhead for holistic view.	Designed for simplicity (reduces burden); aims for efficiency via structured guidance and integration.
Adaptability to Modern Dev (Agile, DevOps, CI/CD)	Less adaptable; static, document-heavy; struggles with rapid iterations/automation.	Well-suited for iterative dev; adopts “documentation as code” principles.	Can integrate for specific functions; generally not holistic for modern pipelines.	Inherently supports Agile, DevOps, CI/CD; promotes “living docs,” versioning, and automation.
Support for Stakeholder Diversity	Often caters to specific roles (analysts); less user-centric.	Primarily for dev teams; less emphasis on comprehensive external stakeholder needs.	Typically serves specific user roles related to the tool’s function.	Designed for all diverse stakeholders (devs, users, managers, maintenance, legal) with tailored deliverables.
Integration and Holistic Approach	Fragmented; documents often standalone, limited cross-referencing.	Decentralized, ad hoc; integration informal/manual.	Siloed solutions; strong in niche but high effort for comprehensive system view.	Unified, holistic framework; integrates documentation into SDLC; emphasizes interconnected artifacts and centralized knowledge.

The information presented in this comparative table is synthesized from a comprehensive analysis of the existing literature on software documentation practices and tools, as detailed in Section 3 (“Related Work and Background”) and the rationale for the PDA-SDD model presented in Section 4.1. Specifically, claims regarding Traditional Documentation Standards are supported by sources such as [1,2,5,11] and the various IEEE/ISO guidelines [18–26]. Insights into Agile/Lightweight Documentation Principles are drawn from discussions on modern development methodologies [53,54,56,57,64,65]. Descriptions of Specialized Documentation Tools are based on their functionalities as discussed in [47–52]. The characteristics and advantages of the PDA-SDD Model are derived from its conceptualization and design presented throughout this paper, particularly in Section 4, building upon the findings of the authors’ systematic review [15].

5. Evaluation

The research methodology utilized a questionnaire, as previously detailed, to gather feedback from 72 software systems engineering specialists and experts on the proposed model and its sub-models. This was administered electronically. The questionnaire was designed to align with the research objectives, specifically focusing on the model’s perceived generality, simplicity, and efficiency.

The questionnaire comprised five main sections. The first gathered general information about the participants, while the second presented a direct question for each of the six sub-models (SRSD, RLD, DDD, CLD, SUMD, and EULA), asking for an evaluation on a five-point scale. The third, fourth, and fifth sections contained a series of statements to assess

the model's generality, simplicity, and efficiency, respectively, with participants indicating their agreement on a five-point scale from “strongly disagree” to “strongly agree.”

Analyzing the results from the second section of the questionnaire (Figures 9–14), the SRSD sub-model elicited predominantly positive responses across all three criteria. Specifically, over 69% of participants assigned high ratings for generality, exceeding 65% for simplicity, and surpassing 55% for efficiency. Turning to the RLD sub-model, the data reveal exceptionally high evaluations, with the efficiency criterion garnering a near-unanimous high rating of approximately 97%, closely followed by simplicity at 92%. In contrast, the DDD sub-model exhibited a greater degree of divergence. Notably, 55% of respondents provided high ratings for simplicity, while only 46% considered its generality and efficiency to warrant high ratings.

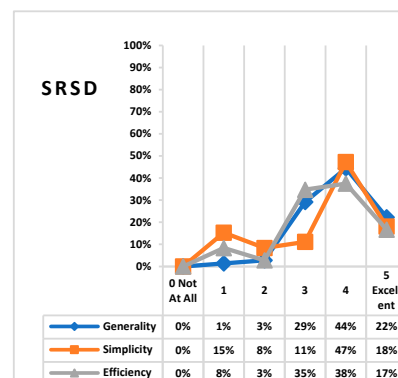


Figure 9. Participants' responses on the generality, simplicity, and efficiency of the SRSD sub-model ($n = 72$).

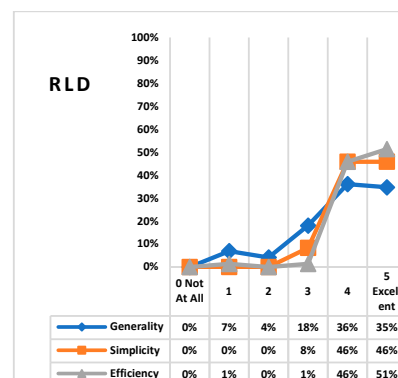


Figure 10. Participants' responses on the generality, simplicity, and efficiency of the RLD sub-model ($n = 72$).

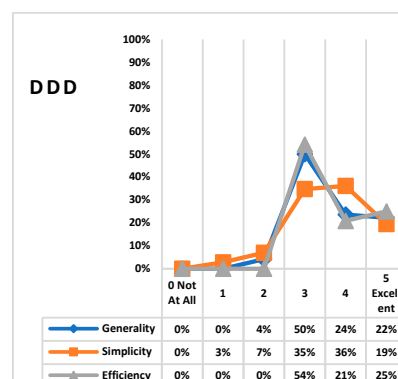


Figure 11. Participants' responses on the generality, simplicity, and efficiency of the DDD sub-model ($n = 72$).

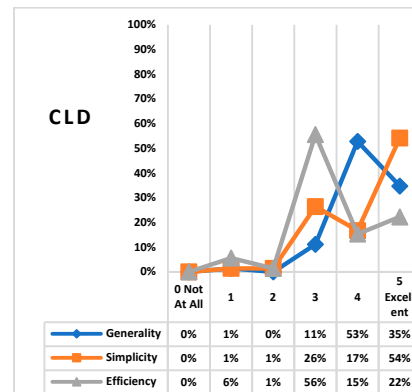


Figure 12. Participants' responses on the generality, simplicity, and efficiency of the CLD sub-model ($n = 72$).

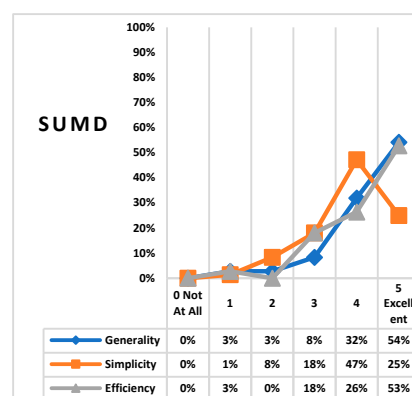


Figure 13. Participants' responses on the generality, simplicity, and efficiency of the SUMD sub-model ($n = 72$).

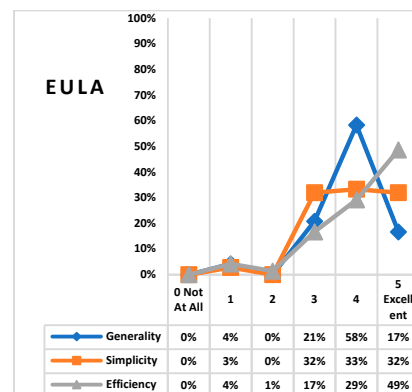


Figure 14. Participants' responses on the generality, simplicity, and efficiency of the EULA sub-model ($n = 72$).

Analysis of the participant evaluations for the CLD sub-model demonstrates a clear trend in perceived characteristics. The simplicity criterion garnered a high level of agreement, with a significant 71% assigning it a positive evaluation. Furthermore, the generality was even more strongly affirmed, with an impressive 88% of participants providing a high rating. In contrast, the efficiency criterion received a more moderate assessment, with approximately 56% of participants assigning a neutral rating (three out of five).

Turning to the SUMD sub-model, the data highlight a prioritization of generality and efficiency in the participants' evaluations. The generality was highly regarded, with 86% of respondents assigning it a high rating, signifying a strong perception of its wide-ranging

applicability. Similarly, the efficiency received a substantial level of positive feedback, with 79% of participants providing a high rating. In comparison, the simplicity received a relatively lower proportion of high ratings at 72%.

Finally, the evaluation of the EULA sub-model reveals a distinct hierarchy in perceived strengths. Efficiency emerged as the most highly rated characteristic with a notable 78% of participants assigning it a positive evaluation, followed by generality with 75%. In contrast, simplicity received the lowest proportion of high ratings at 65%, with a non-negligible 32% of participants assigning a neutral rating.

5.1. In-Depth Analysis of Survey Responses

An in-depth analysis of the responses from the third, fourth, and fifth sections of the questionnaire provides valuable insights into the expert participants' perceptions of the model's scope and applicability.

Among the nine statements designed to evaluate generality, a substantial 92% of participants expressed agreement or strong agreement that the model is suitable for implementation across both small- and large-scale software development projects (Figure 15). Furthermore, a significant majority (exceeding 80%) affirmed that the model could be easily adopted and integrated with existing toolchains. Regarding simplicity, a significant majority (76%) agreed that the model exhibits a commendable level of clarity and conciseness (Figure 16). Additionally, an even more substantial proportion of participants (85%) affirmed that the model effectively supports the integration of visual aids, underscoring its perceived pedagogical value.

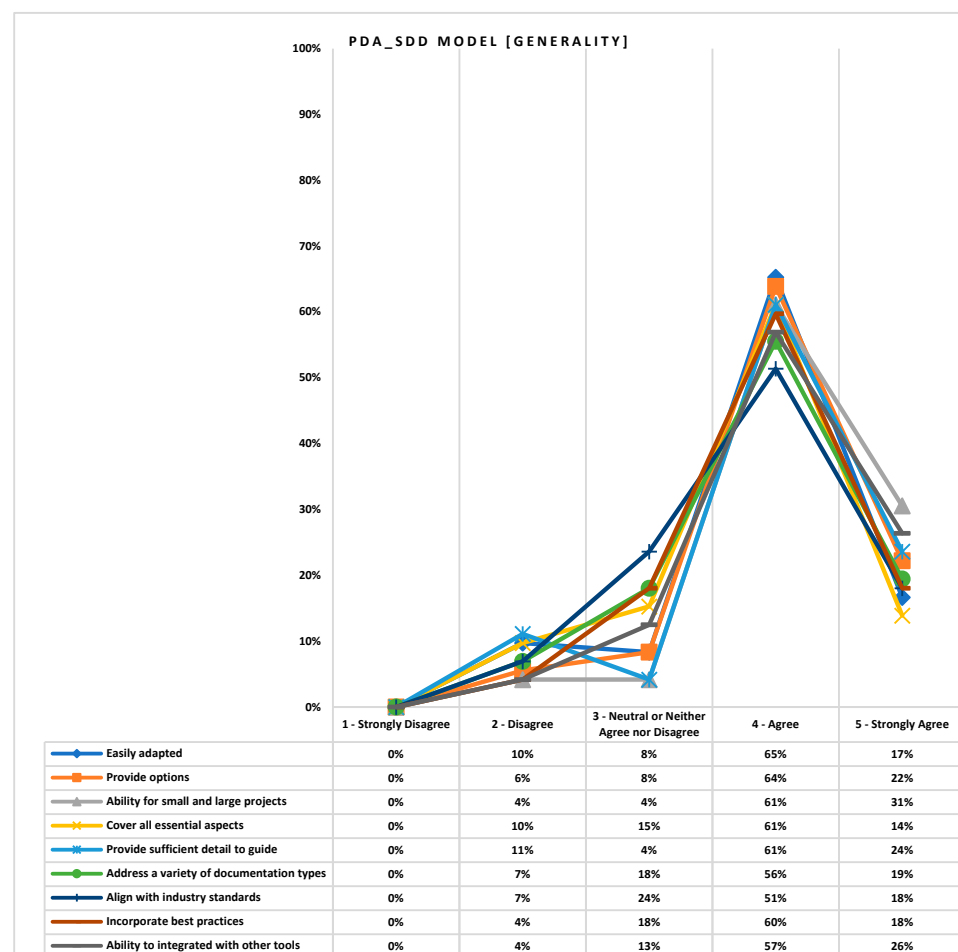


Figure 15. Participants' responses to questions on the generality of the PDA-SDD model ($n = 72$).

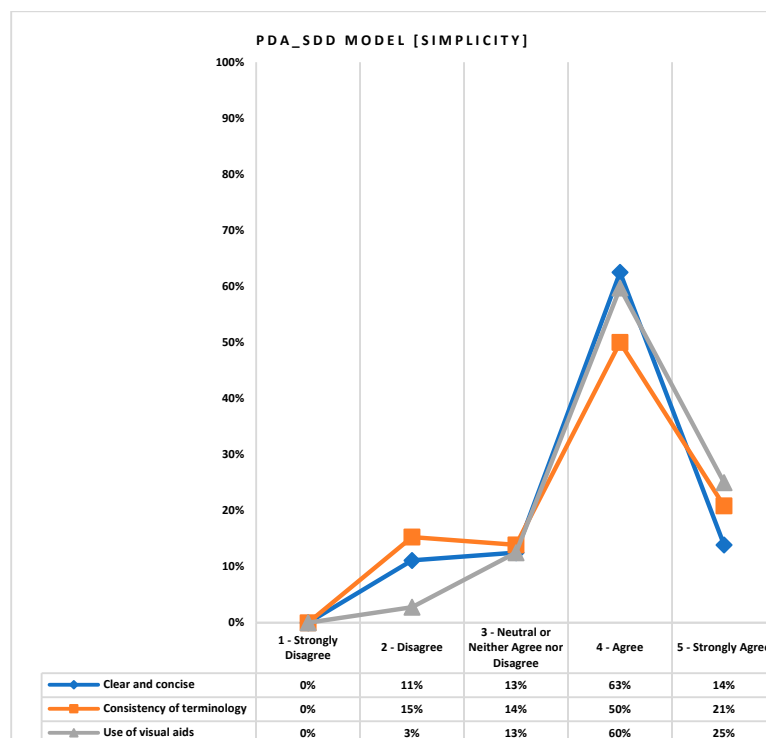


Figure 16. Participants' responses to questions on the simplicity of the PDA-SDD model ($n = 72$).

For efficiency, a significant majority of respondents (82%) expressed agreement or strong agreement that the model enables the creation and management of documentation in an effective and efficient manner (Figure 17). Furthermore, a substantial 78% concurred that the adoption of the model facilitates the subsequent tasks of updating and maintaining documentation.

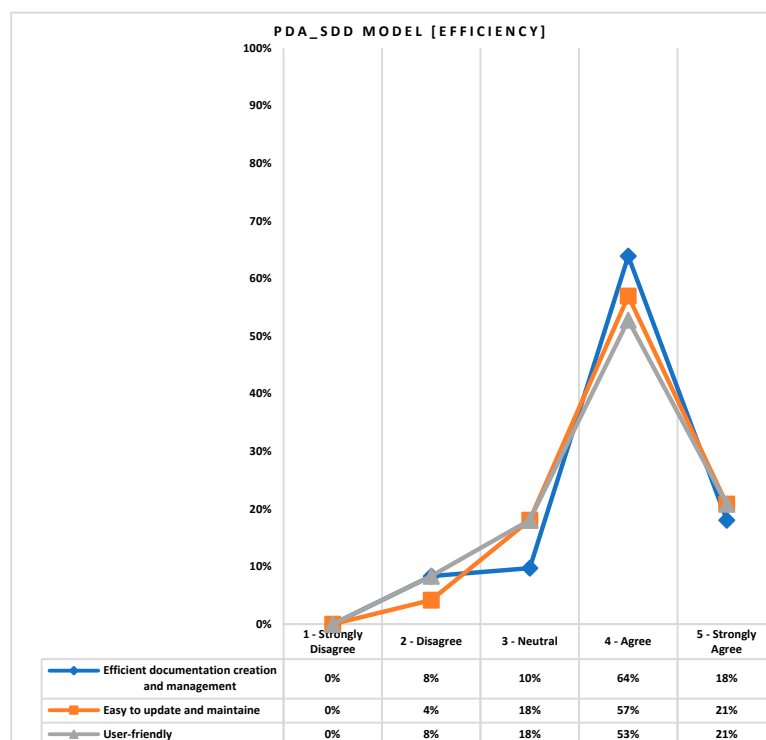


Figure 17. Participants' responses to questions on the efficiency of the PDA-SDD model ($n = 72$).

5.2. Statistical Analysis of Survey Results

To provide a robust quantitative understanding, we performed a detailed statistical analysis of the survey responses from our 72 expert participants. The results consistently show high mean scores, generally above 4.0 on the 5-point Likert scale, across all PDA-SDD attributes. The standard deviations are also notably low, ranging from 0.81 to 1.04. This low variability indicates a strong consensus and high level of agreement among the surveyed experts. The detailed statistical summary for all attributes is presented in Table 4.

Table 4. PDA-SDD model statistical summary.

PDA-SDD Sub-Model/Aspect	Attribute	N (Respondents)	Mean (Avg. Likert Score)	Standard Deviation
[Generality]	Easily adapted	72	4.09	0.92
	Provide options	72	4.17	0.94
	Ability for small and large projects	72	4.29	0.98
	Cover all essential aspects	72	4.21	0.81
	Provide sufficient detail to guide	72	4.19	0.92
	Address a variety of documentation types	72	4.1	0.96
	Align with industry standards	72	3.99	1.04
	Incorporate best practices	72	4.13	0.95
	Ability to integrated with other tools	72	4.13	1.02
[Simplicity]	Clear and concise	72	4.07	0.92
	Consistency of terminology	72	4.06	0.81
	Use of visual aids	72	4.29	0.95
[Efficiency]	Efficient documentation creation and management	72	4.08	0.94
	Easy to update and maintain	72	4	0.93
	User-friendly	72	4.09	0.95

A one-way ANOVA was conducted to compare the mean Likert scores across all attributes within the three main criteria. The results showed no statistically significant difference between the attribute ratings, $F(2, 1077) = 0.70, p = 0.49$. This non-significant result indicates that while there are minor variations in the mean scores, the expert participants' overall perceptions of the model's generality, simplicity, and efficiency are not statistically distinct from each other.

5.3. Discussion of Divergent and Neutral Responses

While the overall results are highly positive, a careful examination of the “neutral” and negative responses provides valuable insight into areas that may warrant further clarification. This deeper analysis is crucial for a transparent account of the study's findings and limitations.

A notable divergence in opinion emerged concerning the model's alignment with prevailing software industry standards. While a respectable 69% of the participants expressed agreement or strong agreement, a substantial 24% adopted a neutral stance. This relatively lower level of direct agreement, coupled with a significant proportion of neutral responses, indicates a perceived ambiguity. It suggests a lack of definitive evidence presented within the model to unequivocally confirm such alignment, warranting further clarification.

A similar trend was observed regarding the consistency of terminology. While a solid majority of 71% of experts agreed, a noteworthy 29% were either neutral or disagreed. This substantial proportion of non-committal or negative responses signals a potential underlying issue with the uniformity and coherence of the language used across the model's various components. Such inconsistencies, even if perceived by a significant minority, could introduce ambiguity and detract from the intended simplicity.

6. Discussion

The evaluation of the PDA-SDD model revealed valuable insights into its perceived utility among software engineering experts. The results show a high degree of positive consensus, particularly regarding the model's generality, simplicity, and efficiency. This is further supported by an illustrative case study (a hypothetical case study) of the Collaborative Project Management Platform (CPMP) and the model's inherent alignment with modern development paradigms.

The CPMP case study demonstrates the PDA-SDD model's practical application across the entire software development lifecycle, highlighting its contribution to enhanced generality, simplicity, and efficiency through its defined documentation artifacts. During the Pre-implementation phase, a Software Requirements Specifications Document (SRSD) and a Resources List Document (RLD) established a foundational blueprint for CPMP, ensuring generality and resource efficiency. In the During-implementation phase, a continuously updated Detailed Design Document (DDD) and a Change Log Document (CLD) fostered technical understanding and efficient change tracking. Finally, the After-implementation phase produced essential artifacts like a Software User Manual and the final Source Code, ensuring simplicity for end-users and generality for future maintainers. The versioning system and centralized data storage throughout the lifecycle further underscored the model's efficiency.

Furthermore, the PDA-SDD model aligns inherently with contemporary software development paradigms, such as Agile methodologies, DevOps practices, and CI/CD pipelines. It supports Agile's iterative nature by treating documents like the SRSD and DDD as living artifacts, continuously refined with each sprint. The Change Log Document (CLD) systematically tracks incremental modifications, ensuring traceability. The model complements DevOps by embedding documentation processes within automated workflows, promoting the automated generation of documentation from elements like the DDD and Source Code. All documentation, managed via PDA-SDD's versioning and centralized storage, can be treated as "code," allowing its integration into CI/CD pipelines for continuous testing and publication. This provides an adaptable framework that positions documentation as a key enabler in contemporary software engineering.

The comprehensive analysis of existing models, which likely informed the development of the PDA-SDD model, suggested the potential for a single framework that is general, simple, and efficient. The evaluation results indicate that while some sub-models (like SRSD and RLD) received high ratings across all three criteria, the overall model presents a mixed picture. No single sub-model or the overall model achieved high ratings across all three dimensions from all participants. Therefore, based on the evaluation of the proposed model, the ideal model that is perfectly general, simple, and efficient across all its components remains a challenge.

7. Limitations

Our survey results offer valuable initial insights into the PDA-SDD model's perceived usefulness and relevance, as assessed by self-reported experts. However, it is crucial to acknowledge the inherent limitations of this current evaluation. Our study used a survey with a Likert scale, which captures subjective perceptions rather than objective, empirical performance data. While these findings reflect participants' theoretical understanding and potential benefits of the model, this evaluation did not include a real-world case study, a pilot implementation in an actual software development environment, or a direct comparison against existing documentation models under similar conditions. Therefore, while the survey provides a foundational understanding of the model's conceptual appeal, it does not offer conclusive empirical evidence of its practical effectiveness, efficiency gains,

or distinct advantages in a live development context. These critical aspects demand more rigorous and robust validation through future research.

8. Conclusion and Future Work

The research aimed to address the persistent challenges in software documentation by investigating the existence and feasibility of a software documentation model that is general, simple, and efficient. The development and evaluation of the proposed PDA-SDD model demonstrate that it is feasible to develop a model aiming for these characteristics. The positive feedback received for several aspects, particularly the generality and efficiency of the SUMD sub-model and the generality and simplicity of the CLD sub-model, suggests that significant progress can be made in this direction. However, the varying perceptions across different sub-models and the specific concerns raised about terminological consistency (simplicity) and user-friendliness (efficiency) indicate that achieving the desired levels across all dimensions remains an ongoing effort requiring further refinement and potentially trade-offs.

The proposed PDA-SDD model shows significant promise in tackling the persistent challenges of software documentation. This study's findings highlight a critical need for further refinement: optimizing the model's simplicity and efficiency across all its components. Addressing concerns about terminological consistency and improving user-friendliness within the model's application should be immediate priorities to encourage wider adoption and seamless integration into development workflows.

Building on the initial validation of PDA-SDD's perceived utility, future research will focus on a more comprehensive and empirically grounded evaluation. A key next step involves conducting real-world case studies and pilot implementations of the PDA-SDD model in diverse software development organizations and projects. These investigations will gather objective data on the model's practical effectiveness, its ability to improve documentation processes, and its adaptability across different team structures and technologies.

Furthermore, future efforts will include rigorous comparative analyses of PDA-SDD against prominent existing documentation frameworks, such as IEEE 830, ISO/IEC 25010, and established agile documentation methods. These comparisons will use controlled experiments or observational studies to objectively benchmark the model's performance, pinpoint its unique advantages, and quantify its benefits in terms of documentation quality, reduced maintenance effort, increased stakeholder satisfaction, and overall project success. Such extensive empirical validations are essential to fully confirm the model's practical impact and solidify its contribution to the field of software documentation.

To further strengthen the empirical foundation of PDA-SDD's evaluation and mitigate the methodological considerations noted previously, subsequent and more advanced empirical studies will also incorporate robust participant validation methods, such as specific screening questions or experience verification criteria, to ensure the reliability of expert classification. Furthermore, these future investigations will strategically employ triangulation techniques, combining multiple data sources and methods, to cross-validate findings and address potential biases like selection and response bias, thereby enhancing the overall trustworthiness and generalizability of the results.

Beyond empirical validation, future research will pursue several specific, interconnected pathways to enhance the model's practical implementation. One vital direction involves continuing to refine specific sub-models within PDA-SDD and exploring why perceptions of the model's attributes might vary. This iterative process ensures each component contributes maximally to overall effectiveness.

Crucially, substantial future work will concentrate on concrete strategies for the model's practical adoption in real-world software engineering environments. This in-

cludes developing specialized tooling designed to directly support PDA-SDD principles, such as automating document generation, facilitating adherence to defined structures, and providing real-time consistency checks for artifacts like the SRSD and DDD. Additionally, creating standardized templates for PDA-SDD's mandated artifacts (e.g., SRSD, RLD, DDD, CLD, and Software User Manuals) would significantly reduce the burden on practitioners, lower adoption barriers, and promote project consistency. Investigating seamless integration with established project management platforms (like Jira, GitHub Projects, Confluence) represents another pivotal step, embedding documentation activities directly within existing developer and project manager workflows to enhance visibility and accessibility of key information.

The positive feedback on PDA-SDD's generality confirms its strong conceptual foundation, affirming its potential to become a widely applicable and versatile documentation framework. This solid foundation provides a clear impetus for continued development, contingent on rigorous improvements in simplicity and efficiency, which will ultimately unlock its full potential to significantly advance software documentation practices.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflict of interest.

Appendix A

Software Requirements Specifications Document (SRSD) <Project Name>

Software Requirements Specifications Document (SRSD)

For

<Project Name>

Prepared by

Date

Versions

Version no.	Title	Main changes	Prepared by	Date

Executive summary <optional>

Contents

1	The Software System	3
1.1	General Info	3
1.1.1	Scope	3
1.1.2	Objectives	3
1.1.3	Main Functions	3
1.2	Functional Requirements (SFR)	3
1.2.1	Input/Output Requirements (SFR-IO)	3
1.2.2	Processing Requirements (SFR-PR)	3
1.2.3	Business Rules (SFR-BR)	4
1.2.4	Security Requirements (SFR-SR)	4
1.2.5	Integration Requirements (SFR-IR)	4
1.3	Non-Functional Requirements (SNFR)	4
1.3.1	Performance (SNFR-P)	4
1.3.2	Usability (SNFR-U)	4
1.3.3	Security (SNFR-S)	4
1.3.4	Reliability (SNFR-R)	4
1.3.5	Maintainability (SNFR-M)	4
2	Appendix A	5
2.1	Requirements coding model	5

Resources List Document (RLD)

For
<Project Name>

Prepared by
.....
Date
.....

Versions

Version no.	Title	Main changes	Prepared by	Date

Executive summary <optional>

Contents

1	Resources.....	3
1.1	Human Resources	3
1.1.1	Human Resources Summary	3
1.1.2	Human Resources Details	3
1.2	Equipments	3
1.2.1	Hardware	3
1.2.2	Software	3

1 Resources

1.1 Human Resources

1.1.1 Human Resources Summary

#	Job Title	Sum

1.1.2 Human Resources Details

#	Name	Job Title	Experience	Qualification	Hourly Cost

1.2 Equipment

1.2.1 Hardware

Category	Sum	Types	Note
Computers			
Servers			
Storage Devices			

1.2.2 Software

#	Category	Sum	Types	Note
1	Development Environment			
2	Version Control System			
3	Database Management System			
4	Operating System			
5	Design Tools			
6	Testing Tools			
7	Deployment Tools			
8	Project Management Tools			
9	Communication Tools			
10	Documentation Tools			
11	Virtual Machines			

Detailed Design Document (DDD)

For
<Project Name>

Versions

Version no.	Title	Main changes	Prepared by	Date

Executive summary <optional>

Contents

1	Traceability Matrix.....	1
2	System Architecture:	2
2.1	High-Level Diagram	2
2.2	Technology Stack	2
3	Data Design.....	2
3.1	Entity-Relationship Diagram (ERD).....	2
3.2	Data Structures	2
3.3	Database Schema	2
4	Interface Design	2
4.1	User Interface (UI) Specifications	2
4.2	API Specifications.....	2
5	Module Design.....	2
5.1	Module <Feature 1>.....	2
5.1.1	Module Responsibilities	3
5.1.2	Module Structure	3
5.1.3	Module Interactions	3
5.1.4	Algorithm Descriptions	3
5.1.5	Data Structure Selection	3
5.2	Module <Feature 2>.....	3
5.3	Module <Feature 3>.....	3
5.4	Module <Feature n>.....	3

1 Traceability Matrix		Requirement Code						
Module ID								

Changes Log Document (CLD)

For
<Project Name>

Executive summary <optional>
.....

Contents

1	Feature 1	2
1.1	Log <Auto-Incremented>	2
1.1.1	Significance.....	2
1.1.2	Results.....	2
1.1.3	Date.....	2
1.1.4	Changes Summary	2
1.1.5	Detailed Changes	2
1.1.6	References.....	2
1.1.7	Issue Tracking System.....	2
1.1.8	Author	2

Software User Manual Document (SUMD)

For
<Project Name>
Prepared by
.....
Date
.....

Executive summary <optional>
.....

Contents

1	Introduction	3
1.1	Overview	3
1.2	Scope	3
1.3	Conventions	3
2	Getting Started	3
2.1	Installation	3
2.2	System Requirements	3
2.3	Activation	3
3	Basic Functionality	3
3.1	User Interface	3
3.2	Core Features	3
3.3	Common Tasks	3
4	Advanced Features	3
4.1	In-depth Explanations	3
5	Troubleshooting	3
5.1	Common Problems	3
5.2	Error Messages	3
5.3	Technical Support	3
6	Glossary	3
6.1	Definitions	4
7	Appendix	4
7.1	Additional Information	4

End-User License Agreement (EULA)

For
<Project Name>
Prepared by
.....
Date
.....

Contents

1	Grant of License	3
2	Ownership	3
3	Restrictions	3
4	Warranty Disclaimer	3
5	Indemnification	3
6	Termination	3
7	Governing Law and Jurisdiction	3
8	Entire Agreement	3
9	Severability	3

References

1. Chomal, V.S.; Saini, J.R. Software project documentation-an essence of software development. *Int. J. Adv. Netw. Appl.* **2015**, *6*, 2563.

2. Barker, T.T. Introduction: Research sources in software documentation. In *Perspectives on Software Documentation*; Routledge: Oxfordshire, UK, 2020; pp. 7–21. Available online: <https://www.taylorfrancis.com/chapters/edit/10.4324/9781315223919-1/introduction-research-sources-software-documentation-thomas-barker> (accessed on 9 September 2024).

3. Satish, C.J.; Anand, M. Software documentation management issues and practices: A survey. *Indian J. Sci. Technol.* **2016**, *9*, 1–7. [CrossRef]

4. Manai, O. How Software Documentation Helps Communication in Development Teams: A Case Study on Architecture and Design Documents. 2019. Available online: <https://gupea.ub.gu.se/handle/2077/62545> (accessed on 9 September 2024).

5. Shirk, H.N. Prologue for teaching software documentation. In *Perspectives on Software Documentation*; Routledge: Oxfordshire, UK, 2020; pp. 25–44. Available online: <https://www.taylorfrancis.com/chapters/edit/10.4324/9781315223919-3/prologue-teaching-software-documentation-henrietta-nickels-shirk> (accessed on 9 September 2024).

6. Parnas, D.L. Precise Documentation: The Key to Better Software. In *The Future of Software Engineering*; Nanz, S., Ed.; Springer: Berlin/Heidelberg, Germany, 2011; pp. 125–148. [CrossRef]

7. Cohen, N.E. Problems of form in software documentation. In *Perspectives on Software Documentation*; Routledge: Oxfordshire, UK, 2020; pp. 123–136. Available online: <https://www.taylorfrancis.com/chapters/edit/10.4324/9781315223919-9/problems-form-software-documentation-nancy-cohen> (accessed on 9 September 2024).

8. Plösch, R.; Dautovic, A.; Saft, M. The value of software documentation quality. In Proceedings of the 2014 14th International Conference on Quality Software, Allen, TX, USA, 2–3 October 2014; IEEE: New York, NY, USA, 2014; pp. 333–342. Available online: <https://ieeexplore.ieee.org/abstract/document/6958422/> (accessed on 9 September 2024).

9. De Boer, R.C.; Van Vliet, H. Writing and reading software documentation: How the development process may affect understanding. In Proceedings of the 2009 ICSE Workshop on Cooperative and Human Aspects on Software Engineering, Vancouver, BC, Canada, 17 May 2009; IEEE: New York, NY, USA, 2009; pp. 40–47. Available online: <https://ieeexplore.ieee.org/abstract/document/5071409/> (accessed on 9 September 2024).

10. Aghajani, E.; Nagy, C.; Vega-Marquez, O.L.; Linares-Vasquez, M.; Moreno, L.; Bavota, G.; Lanza, M. Software documentation issues unveiled. In Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), Montreal, QC, Canada, 25–31 May 2019; IEEE: New York, NY, USA, 2019; pp. 1199–1210. Available online: <https://ieeexplore.ieee.org/abstract/document/8811931/> (accessed on 24 April 2024).

11. Ramesh, M.R.; Reddy, C.S. Metrics for software requirements specification quality quantification. *Comput. Electr. Eng.* **2021**, *96*, 107445. [CrossRef]
12. Ali, S.W.; Ahmed, Q.A.; Shafi, I. Process to enhance the quality of software requirement specification document. In Proceedings of the 2018 International Conference on Engineering and Emerging Technologies (ICEET), Lahore, Pakistan, 22–23 February 2018; IEEE: New York, NY, USA, 2018; pp. 1–7. Available online: <https://ieeexplore.ieee.org/abstract/document/8338619/> (accessed on 9 September 2024).
13. Ali, N.; Lai, R. A method of software requirements specification and validation for global software development. *Requir. Eng.* **2017**, *22*, 191–214. [CrossRef]
14. Oo, K.H.; Nordin, A.; Ismail, A.R.; Sulaiman, S. An analysis of ambiguity detection techniques for software requirements specification (SRS). *Int. J. Eng. Technol.* **2018**, *7*, 501–505. [CrossRef]
15. Alzahrani, A.A.H. Software Systems Documentation: A Systematic Review. *Int. J. Adv. Comput. Sci. Appl.* **2024**, *15*, 155. [CrossRef]
16. Theunissen, T.; Hoppenbrouwers, S.; Overbeek, S. Approaches for documentation in continuous software development. *Complex Syst. Inform. Model. Q.* **2022**, *32*, 1–27. [CrossRef]
17. Theunissen, T.; Hoppenbrouwers, S.; Overbeek, S. Evaluation of Approaches for Documentation in Continuous Software Development. 2023. Available online: <https://repository.ubn.ru.nl/bitstream/handle/2066/293197/293197.pdf?sequence=1> (accessed on 8 September 2024).
18. Chikh, A.; Aldayel, M. A new traceable software requirements specification based on IEEE 830. In Proceedings of the 2012 International Conference on Computer Systems and Industrial Informatics, Sharjah, United Arab Emirates, 18–20 December 2012; IEEE: New York, NY, USA, 2012; pp. 1–6. Available online: <https://ieeexplore.ieee.org/abstract/document/6454481/> (accessed on 8 September 2024).
19. Chikh, A.; Aldayel, M. Reengineering Requirements Specification Based on IEEE 830 Standard and Traceability. In *New Perspectives in Information Systems and Technologies*; Rocha, Á., Correia, A.M., Felix, B.T., Stroetmann, K.A., Eds.; Advances in Intelligent Systems and Computing; Springer International Publishing: Cham, Switzerland, 2014; Volume 275, pp. 211–227. [CrossRef]
20. Pena, I.; Martinez-Anido, C.B.; Hodge, B.-M. An extended IEEE 118-bus test system with high renewable penetration. *IEEE Trans. Power Syst.* **2017**, *33*, 281–289. [CrossRef]
21. Handoyo, E.; Isnantoa, R.R.; Sonda, M.A. SRS Document Proposal Analysis on the Design of Management Information Systems According to IEEE STD 830-1998. *Procedia-Soc. Behav. Sci.* **2012**, *67*, 123–134. [CrossRef]
22. Villamizar, L.A.E.; Sanchez-Segura, M.; de Amescua, A.; García, L. The Project Management in the Development Process. In *Innovations Through Information Technology*; Idea Group Publishing: Hershey, PA, USA, 2004. Available online: <https://www.academia.edu/download/86832230/32387.pdf> (accessed on 9 September 2024).
23. Codur, K.B.; Dogru, A.H. Regulations and software evolution: An example from the military domain. *Sci. Comput. Program.* **2012**, *77*, 636–643. [CrossRef]
24. Dimeska, K.; Savoska, S. Model of software development using RAD methods and standard ISO/IEC 12207. In Proceedings of the 8th International Conference on Applied Internet and Information Technologies, Bitola, Macedonia, 1 July–5 October 2018; pp. 48–51. Available online: https://eprints.uklo.edu.mk/8750/1/Proceedings_AIIT2018.pdf#page=66 (accessed on 9 September 2024).
25. Estdale, J.; Georgiadou, E. Applying the ISO/IEC 25010 Quality Models to Software Product. In *Systems, Software and Services Process Improvement*; Larrucea, X., Santamaria, I., O'Connor, R.V., Messnarz, R., Eds.; Communications in Computer and Information Science; Springer International Publishing: Cham, Switzerland, 2018; Volume 896, pp. 492–503. [CrossRef]
26. Hussain, A.; Mkpojiogu, E.O. An application of the ISO/IEC 25010 standard in the quality-in-use assessment of an online health awareness system. *J. Teknol.* **2015**, *77*, 9–13. [CrossRef]
27. Acharya, A.; Sinha, D. Assessing the quality of m-learning systems using ISO/IEC 25010. *Int. J. Adv. Comput. Res.* **2013**, *3*, 67.
28. Arcos-Medina, G.; Mauricio, D. The influence of the application of agile practices in software quality based on ISO/IEC 25010 standard. *Int. J. Inf. Technol. Syst. Approach* **2020**, *13*, 27–53. [CrossRef]
29. Falco, M.; Robiolo, G. Building a catalogue of ISO/IEC 25010 quality measures applied in an industrial context. *J. Phys. Conf. Ser.* **2021**, *1828*, 012077. [CrossRef]
30. Gustriansyah, R.; Suhandi, N.; Alie, J.; Antony, F.; Heryati, A. Optimization of laboratory application by utilizing the ISO/IEC 25010 model. *Iop Conf. Ser. Mater. Sci. Eng.* **2021**, *1088*, 012067. [CrossRef]
31. Sánchez, M. Assessing the quality of MOOC using ISO/IEC 25010. In Proceedings of the 2016 XI Latin American Conference on Learning Objects and Technology (LACLO), San Carlos, Costa Rica, 3–7 October 2016; IEEE: New York, NY, USA, 2016; pp. 1–4. Available online: <https://ieeexplore.ieee.org/abstract/document/7751803/> (accessed on 9 September 2024).
32. Ouhbi, S.; Idri, A.; Fernández-Alemán, J.L.; Toval, A.; Benjelloun, H. Applying ISO/IEC 25010 on mobile personal health records. In Proceedings of the International Conference on Health Informatics, Lisbon, Portugal, 12–15 January 2015; SciTePress: Setúbal, Portugal, 2015; pp. 405–412. Available online: <https://www.scitepress.org/Papers/2015/52166/> (accessed on 9 September 2024).

33. Peters, E.; Aggrey, G.K. An ISO 25010 based quality model for ERP systems. *Adv. Sci. Technol. Eng. Syst. J.* **2020**, *5*, 578–583. [CrossRef]
34. Saptarini, I.; Rochimah, S.; Yuhana, U.L. Security Quality Measurement Framework for Academic Information System (AIS) Based on ISO/IEC 25010 Quality Model. *Iptek J. Proc. Ser.* **2017**, *3*, 128–135. [CrossRef]
35. ISO/IEC 15288; Information Technology—Life Cycle Management—System Life Cycle Processes. International Organization for Standardization/International Electrotechnical Commission: Geneva, Switzerland, 2002.
36. ISO/IEC 25000; Software Engineering—Software Product Quality Requirements and Evaluation (SQuaRE)—Guide to SQuaRE. International Organization for Standardization/International Electrotechnical Commission: Geneva, Switzerland, 2005.
37. ISO/IEC 25001; Software Engineering—Software Product Quality Requirements and Evaluation (SQuaRE)—Planning and Management. International Organization for Standardization/International Electrotechnical Commission: Geneva, Switzerland, 2007.
38. ISO/IEC 25010; Software Engineering—System and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Model. International Organization for Standardization/International Electrotechnical Commission: Geneva, Switzerland, 2011.
39. ISO/IEC 25012; Software Engineering—Software Product Quality Requirements and Evaluation (SQuaRE)—Data Quality Model. International Organization for Standardization/International Electrotechnical Commission: Geneva, Switzerland, 2006.
40. ISO/IEC 25020; Software Engineering—Software Product Quality Requirements and Evaluation (SQuaRE)—Measurement Reference Model and Guide. International Organization for Standardization/International Electrotechnical Commission: Geneva, Switzerland, 2007.
41. IEEE 830; IEEE Standards Association. IEEE: New York, NY, USA, 1993. Available online: <https://standards.ieee.org/ieee/830/1222/> (accessed on 8 September 2024).
42. IEEE 12207; IEEE Standards Association. IEEE: New York, NY, USA, 1996. Available online: <https://standards.ieee.org/ieee/12207-2/10353/> (accessed on 8 September 2024).
43. IEEE 610.12; IEEE Standard Glossary of Software Engineering Terminology. IEEE: New York, NY, USA, 1990.
44. Vendome, C. Assisting Software Developers with License Compliance. 2018. Available online: <https://scholarworks.wm.edu/etd/1550153779/> (accessed on 9 September 2024).
45. Open Source Initiative. Available online: <https://opensource.org> (accessed on 9 September 2024).
46. The GNU Operating System and the Free Software Movement. Available online: <https://www.gnu.org/> (accessed on 9 September 2024).
47. IBM. IBM Engineering Requirements Management DOORS Next. IBM.com. Available online: <https://www.ibm.com/products/requirements-management-doors-next> (accessed on 5 June 2025).
48. Abbas, W.; Butt, W.H. Systematic Literature Review on Requirement Management Tools. In Proceedings of the 2022 International Conference on Emerging Trends in Smart Technologies (ICETST), Karachi, Pakistan, 23–24 September 2022; pp. 1–6. [CrossRef]
49. GitBook. Official Documentation and Features Overview. GitBook.com. Available online: <https://www.gitbook.com/> (accessed on 5 June 2025).
50. Mens, T.; Van Gorp, J.; Van Den Brandt, D. Documentation as code: A state of the practice review. In Proceedings of the 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), Campobasso, Italy, 20–23 March 2018; pp. 599–603.
51. Stoplight. API Design and Documentation Platform. Stoplight.io. Available online: <https://stoplight.io/> (accessed on 5 June 2025).
52. Madni, A.M.; Sievers, M. Model-based systems engineering: Motivation; current capabilities. *Syst. Eng.* **2018**, *21*, 2530–2541. [CrossRef]
53. Johan, E.J.; Yusoff, S.A.M.; Mohammad, W.A.W.; Mydin, A.M. The impact of AI tools on software development practices and programmer productivity. *Beyond Boundaries Multidimens. Horiz. E-Learn.* **2025**, *9*, 1–7.
54. Bernardo, J.H.; da Costa, D.A.; Cogo, F.R.; de Medeiros, S.Q.; Kulesza, U. Continuous Integration Practices in Machine Learning Projects: The Practitioners’ Perspective. *arXiv* **2025**, arXiv:2502.17378. [CrossRef]
55. Birru, H.; Cicchetti, A.; Latifaj, M. Supporting Automated Documentation Updates in Continuous Software Development with Large Language Models. In Proceedings of the 20th International Conference on Evaluation of Novel Approaches to Software Engineering, Porto, Portugal, 4–6 April 2025.
56. Pastrana, M.; Ordoñez, H.; Cobos-Lozada, C.A.; Muñoz, M. Best Practices Evidenced for Software Development Based on DevOps and Scrum: A Literature Review. *Appl. Sci.* **2025**, *15*, 5421. [CrossRef]
57. Fawzy, A.; Tahir, A.; Galster, M.; Liang, P. Exploring data management challenges and solutions in agile software development: A literature review and practitioner survey. *Empir. Softw. Eng.* **2025**, *30*, 77. [CrossRef]
58. Smith, T. Morgan Stanley Open Sources CALM: The Architecture as Code Solution Transforming Enterprise DevOps. DevOps.com. Available online: <https://devops.com/morgan-stanley-open-sources-calm-the-architecture-as-code-solution-transforming-enterprise-devops/> (accessed on 26 August 2025).

59. Alenezi, M.; Akour, M. AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions. *Appl. Sci.* **2025**, *15*, 1344. [CrossRef]
60. Foster, R. The Complete Guide to AI in Software Development (2025) | KPIs, ROI & Trends. Available online: <https://emmo.net.co/articles/post/the-complete-guide-to-ai-in-software-development-transforming-code-creation-in-2025.html> (accessed on 26 August 2025).
61. Trinkenreich, B.; Calefato, F.; Hanssen, G.; Blincoe, K.; Kalinowski, M.; Pezzè, M.; Tell, P.; Storey, M.-A. Get on the Train or be Left on the Station: Using LLMs for Software Engineering Research. In Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, Trondheim, Norway, 23–28 June 2025; ACM: New York, NY, USA, 2025; pp. 1503–1507. [CrossRef]
62. Maayan, G.D. Documentation in Agile: Challenges and Trends in 2023—DevOps.com. Available online: <https://devops.com/documentation-in-agile-challenges-and-trends-in-2023/> (accessed on 26 August 2025).
63. Dugbartey, A.N.; Kehinde, O. Optimizing project delivery through agile methodologies: Balancing speed, collaboration and stakeholder engagement. *World J. Adv. Res. Rev.* **2025**, *25*, 1237–1257. [CrossRef]
64. El Aouni, F.; Moumane, K.; Idri, A.; Najib, M.; Jan, S.U. A systematic literature review on Agile, Cloud, and DevOps integration: Challenges, benefits. *Inf. Softw. Technol.* **2024**, *177*, 107569. [CrossRef]
65. Fan, A.; Gokkaya, B.; Harman, M.; Lyubarskiy, M.; Sengupta, S.; Yoo, S.; Zhang, J.M. Large language models for software engineering: Survey and open problems. In Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), Melbourne, Australia, 14–20 May 2023; IEEE: New York, NY, USA, 2023; pp. 31–53.
66. Humble, J.; Farley, D. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*; Pearson Education: London, UK, 2010.
67. Saleh, S.M.; Madhavji, N.; Steinbacher, J. A Systematic Literature Review on Continuous Integration and Deployment (CI/CD) for Secure Cloud Computing. In Proceedings of the 20th International Conference on Web Information Systems and Technologies, Porto, Portugal, 17–19 November 2024.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.